Aula 10 — Continuidade, Disaster Recovery & Arquitetura Final

# E se a região inteira da Tech Pix cair agora?

Hoje a gente troca a promessa de “nunca ficar offline” por metas claras, riscos conhecidos e evidências medidas.

RTO e RPO | Fencing | Warm standby | Reconciliação | Runbook | Game Day

# Ao final desta aula, vocês vão conseguir

1. **Definir RTO, RPO e SLO por capacidade**
   e explicar por que cada capacidade pede metas diferentes
2. **Escolher uma estratégia de recuperação**
   de backup e restore até ativo-ativo, pesando custo e risco
3. **Recuperar sem perder a integridade financeira**
   fencing contra split-brain e reconciliação antes de reabrir
4. **Provar a recuperação com evidências**
   runbook, Game Day e uma proposta defensável para os investidores

ROTEIRO DA AULA

1 · Pressão de negócio | 2 · Incidente | 3 · Objetivos | 4 · Recuperação | 5 · Estratégias e custo | 6 · Integridade financeira | 7 · Runbook e Game Day

# As tecnologias que passamos — e por que cada uma entrou

| CAMADA | TECNOLOGIAS | PROBLEMA QUE RESOLVEU |
|---|---|---|
| **Aplicação** | Java 21 · Spring Boot 3.5 · Flyway · ArchUnit · Testcontainers | Fronteiras de módulo verificadas por teste |
| **Dados e eventos** | PostgreSQL · Kafka · Outbox · Idempotência · Saga | Cada serviço dono do seu dado, fatos compartilhados |
| **Plataforma** | Docker Compose · Kubernetes (kind) · HPA · Kustomize · Argo CD · Gitea · Istio | Cada serviço sobe sozinho, e o ambiente rodando é sempre igual ao que está no Git |
| **Segurança** | Tekton · SonarQube · OWASP ZAP · JFrog Artifactory · Keycloak · OPA / Rego | Só artefato aprovado chega; quem é × o que pode |
| **Observabilidade** | Micrometer · Prometheus · Grafana · Tempo (OTLP) · k6 | Ver falha parcial antes do cliente |
| **Continuidade · hoje** | Replicação streaming · pg_basebackup · Promote + fencing · Reconciliação | E se perdermos a região inteira? |

- Nenhuma ferramenta entrou por moda: cada uma pagou por um problema que a etapa anterior deixou visível.

# Crescer muda o tamanho de cada falha

MAIS CLIENTES E PAGAMENTOS EXIGEM MAIS…

**Capacidade**
Picos, filas e bancos dimensionados para o volume real

**Suporte**
Cada incidente vira chamados, contestações e explicações

**Operação**
Plantão, runbooks e decisões sob pressão

**Confiança**
Clientes, parceiros e reguladores esperam previsibilidade

A MESMA FALHA DE 10 MINUTOS — EXEMPLO DIDÁTICO

| VOLUME HIPOTÉTICO | OPERAÇÕES AFETADAS |
|---|---|
| 100 / min | 1.000 |
| 1.000 / min | 10.000 |
| 10.000 / min | 100.000 |

Multiplicação simples: volume × duração. Não são métricas da Tech Pix.

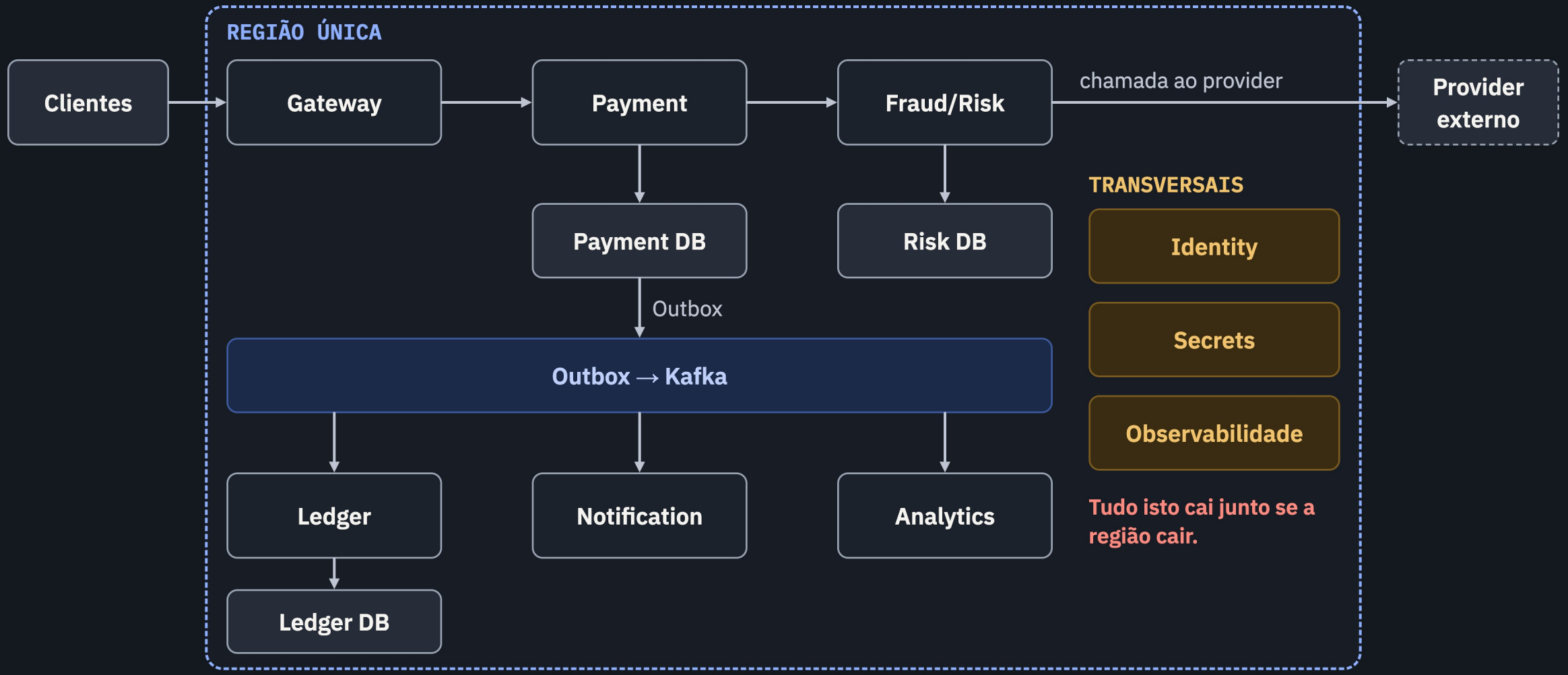
**Quanto maior o negócio, mais operações e pessoas cada falha alcança.**

# O que já sabemos proteger — e onde isso para

| PROBLEMA | O QUE USAMOS | LIMITE SE PERDERMOS A INFRAESTRUTURA |
|---|---|---|
| Evento perdido entre banco e mensageria | **Outbox + Kafka** | Se banco ou cluster somem, o Outbox e o log somem junto |
| Retentativa gerando efeito duplicado | **Idempotência** | Só protege se a chave existir no estado recuperado |
| Acesso indevido entre serviços | **Identity, mTLS, secrets** | Identity ou secrets indisponíveis bloqueiam todos os serviços |
| Falha parcial invisível | **Traces, métricas, logs** | Observabilidade na mesma região fica cega junto |
| Pagamento em UNKNOWN: enviado ao provider, sem resposta — não sabemos se foi pago | **Reconciliação** | Exige estado local confiável para comparar com o provider |
| Dependência lenta ou instável | **Timeout, retry, circuit breaker, bulkhead** | Contêm degradação; não substituem uma região perdida |

- Esses mecanismos tratam falhas parciais. Todos assumem que a infraestrutura onde rodam continua existindo.

# Onde estamos hoje: tudo numa região só



# “Estamos crescendo. Nossos clientes precisam pagar a qualquer hora. Não podemos ficar fora do ar nunca.”

— Investidores da Tech Pix · uma provocação, não um requisito

**Qual capacidade?** | **Para quais clientes?** | **Contra quais falhas?** | **Com qual orçamento?**

DISPONIBILIDADE ABSOLUTA NÃO PODE SER GARANTIDA.
PRECISAMOS DEFINIR:

**Escopo** **SLOs** **RTO** **RPO** **Riscos aceitos**

≠ **Serviço respondendo**

✓ **Pagamento concluído corretamente**

Efeito financeiro único, conhecido e registrado no Ledger.

# Cada nove a mais encolhe a margem de erro

EQUIVALÊNCIA EM 30 DIAS (43.200 MIN)

CÁLCULO, NÃO GARANTIA

| DISPONIBILIDADE | INDISPONIBILIDADE EQUIVALENTE |
|---|---|
| 99,9% | 43 min 12 s |
| 99,99% | ≈ 4 min 19 s |
| 99,999% | ≈ 26 s |

SLO por operações tem outra conta: **bem-sucedidas ÷ válidas**. 99,9% = no máximo 1 operação ruim a cada 1.000, sem falar em minutos.

METAS MAIS EXIGENTES TENDEM A PEDIR

QUALITATIVO

**99,9%**
Redundância básica, alertas, restore testado

**99,99%**
+ failover automatizado, plantão treinado, Game Days

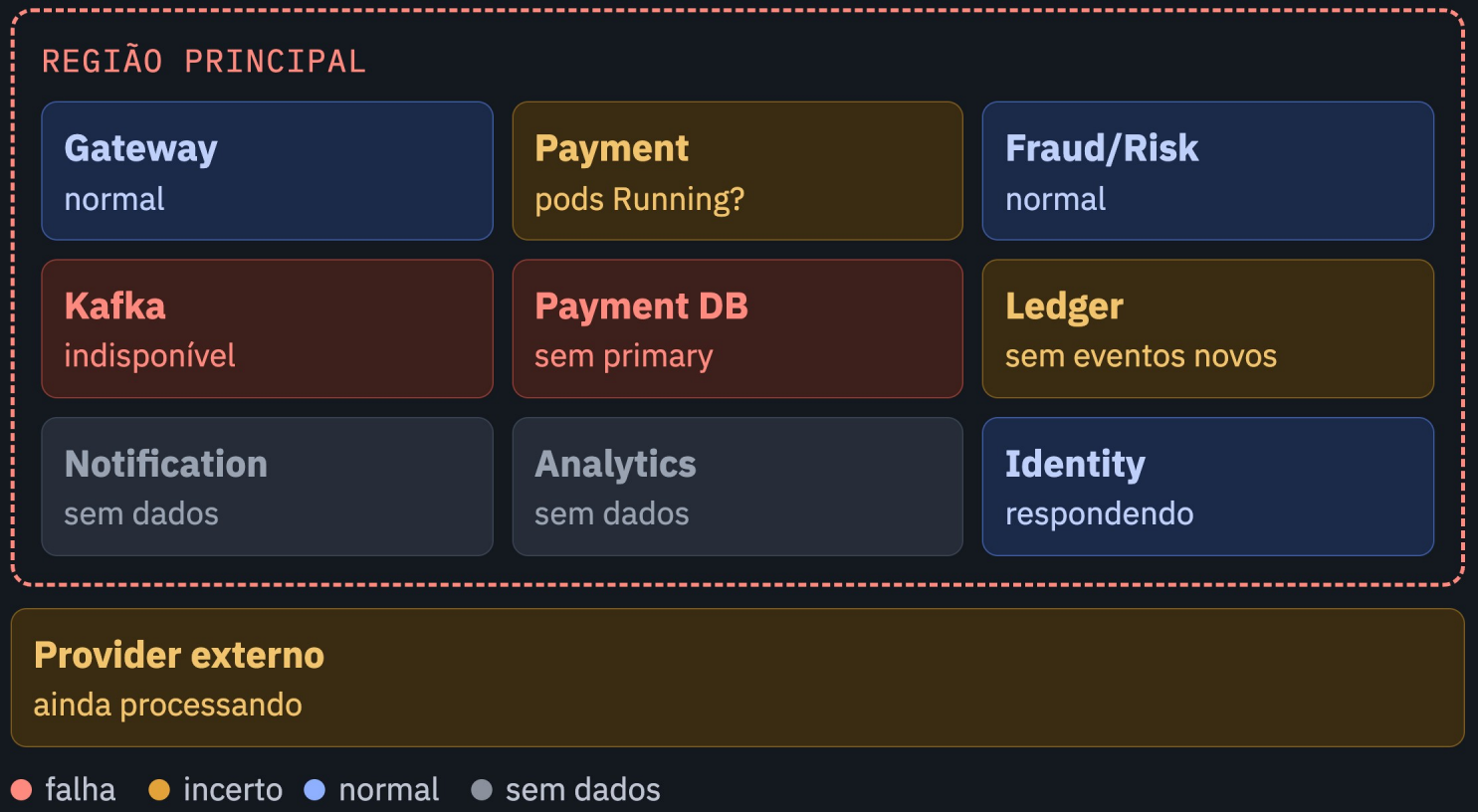
**99,999%**
+ múltiplas regiões, detecção em segundos, disciplina contínua

Tamanho ilustrativo: não é curva de preço.

# Sexta-feira, 18h. A região principal está falhando.

- Tráfego em 2× o usual
- **Payment sem banco primário**
- **Kafka indisponível**
- Alguns pods em Running
- Provider ainda processando
- Clientes repetindo tentativas

**Vocês fazem failover agora?**



# Até onde cada falha se propaga?

HIPÓTESE DIDÁTICA DE DEPENDÊNCIAS

| CAPACIDADE | Banco | Kafka | Identity | Secrets | Provider | Volta quando? |
|---|---|---|---|---|---|---|
| **Movimentação financeira** | ● | ○ | ● | ● | ● | ? |
| **Ledger** | ● | ● | ● | ● | – | ? |
| **Estado dos pagamentos** | ● | ○ | ○ | ● | ○ | ? |
| **Fraud/Risk** | ● | ○ | ● | ● | – | ? |
| **Notification** | ○ | ● | ○ | ● | – | ? |
| **Analytics** | – | ● | ○ | ○ | – | ? |

● dependência direta ○ indireta ou parcial – sem dependência relevante

**O que volta primeiro?**

**O que pode esperar?**

**Até onde a falha se espalha?**

# RPO olha para trás. RTO olha para frente.

## RPO

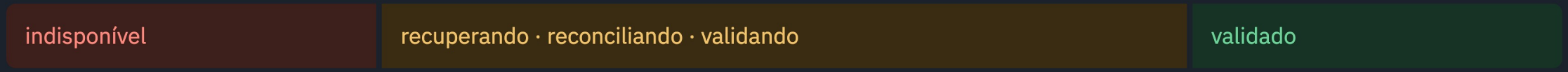
Até onde no passado aceitamos voltar? É a perda máxima de dados, medida em tempo.




17h30 · último ponto recuperável — 18h00 · falha

## RTO

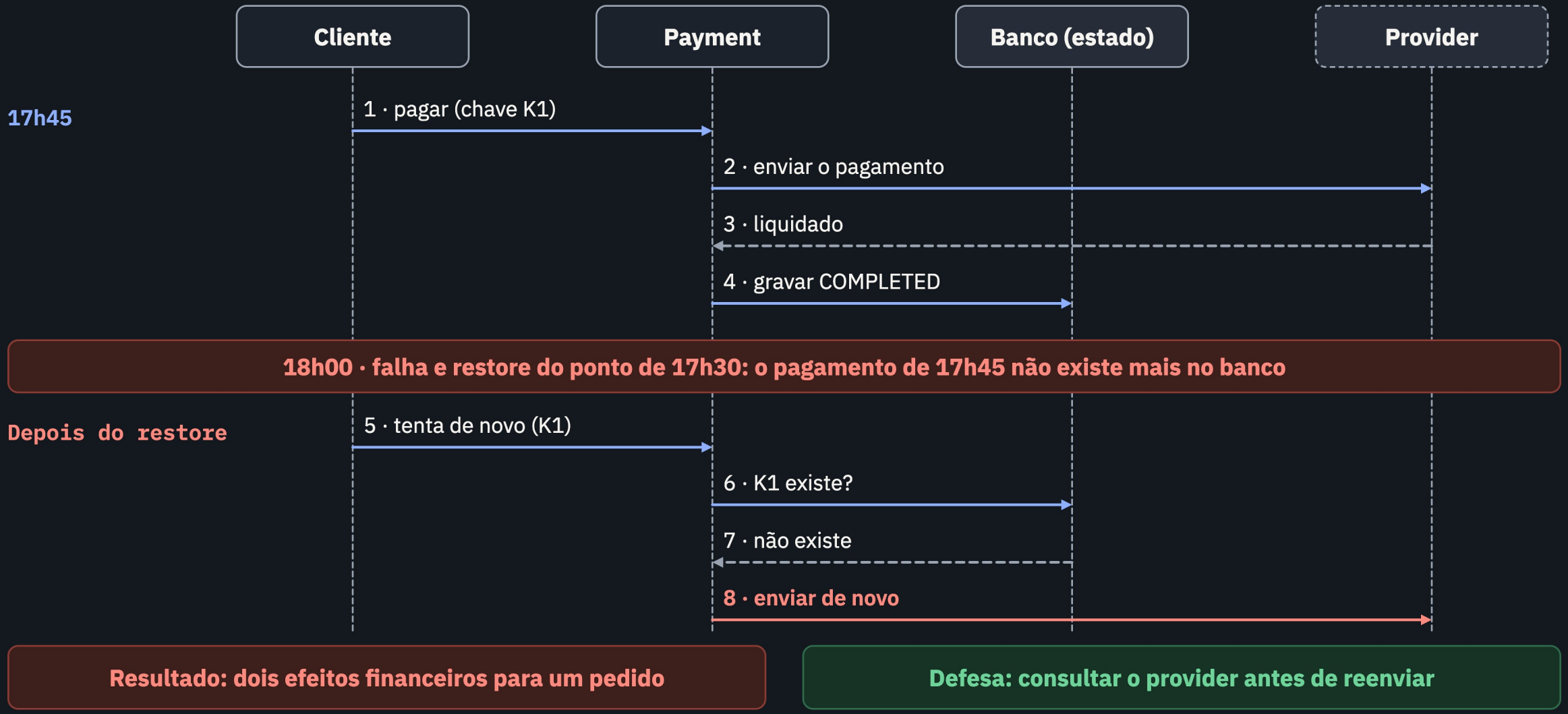
Em quanto tempo precisa estar de pé de novo? É o tempo máximo até a capacidade voltar — validada.




18h00 · falha — 18h00 + RTO

**RISCO** Entre 17h30 e 18h00 o provider pode ter executado pagamentos que **não existem no estado recuperado**. O dinheiro se moveu; nosso banco não sabe.

# Retentativa após restore = pagamento duplicado



# Nem tudo precisa voltar com a mesma pressa

HIPÓTESES DIDÁTICAS

Cada parte do sistema ganha sua própria meta de **RTO** (em quanto tempo volta) e **RPO** (quanto dado aceitamos perder), de acordo com o estrago que causa quando para.

| PARTE DO SISTEMA | RTO · VOLTA EM | RPO · PERDE ATÉ | SE PARAR… |
|---|---|---|---|
| **Payment** faz o pagamento | **5 min** | *30 min* | O cliente não consegue pagar, e as retentativas podem duplicar |
| **Ledger** registra saldos e lançamentos | **5 min** | *30 min* | Ficamos sem registro confiável do dinheiro |
| **Notification** avisa o cliente | **2 h** | *5 horas* | O cliente fica sem aviso e liga para o suporte |
| **Analytics** gera relatórios | **24 h** | *30 horas* | Relatórios atrasam, mas os dados podem ser refeitos a partir dos eventos |

**Quanto custaria exigir 5 minutos para todas?**

# Ter backup não comprova que conseguimos recuperar

| 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|---|---|---|---|---|---|---|
| Obter backup | Restaurar | Recuperar dependências | Validar schema e configuração | Verificar integridade | Reconciliar | Liberar tráfego |

### Banco iniciado

Processo aceita conexões. Dados ainda não verificados.

### Capacidade financeira recuperada

Estados conhecidos, sem duplicação, Ledger consistente.

TESTE PERIÓDICO

### É um simulado de incêndio

Ninguém descobre se a saída de emergência funciona no dia do incêndio. Com o banco é igual: restaurar de verdade, em dia marcado, e cronometrar.

Último tempo medido: **[resultado do último teste]**

**Compare o RTO com o tempo medido do restore completo — não com a estimativa.**

# Promover a réplica: com qual garantia?

**Primary**
indisponível




**Réplica**
atraso conhecido: [medido]

O atraso no momento da falha é a sua exposição — se a réplica for assíncrona.

| | SÍNCRONA | ASSÍNCRONA |
|---|---|---|
| **Confirmação da escrita** | Após a réplica confirmar o recebimento | Após o primary gravar |
| **Latência** | Maior; cresce com a distância | Menor |
| **Disponibilidade** | Réplica fora pode bloquear escritas, conforme a configuração | Primary segue sem depender da réplica |
| **Exposição à perda** | Menor para escritas confirmadas, se a promovida for a síncrona | Escritas dentro do atraso podem se perder |

As garantias dependem da configuração e de **qual réplica** é promovida.

**Vocês promovem? Com qual evidência?**

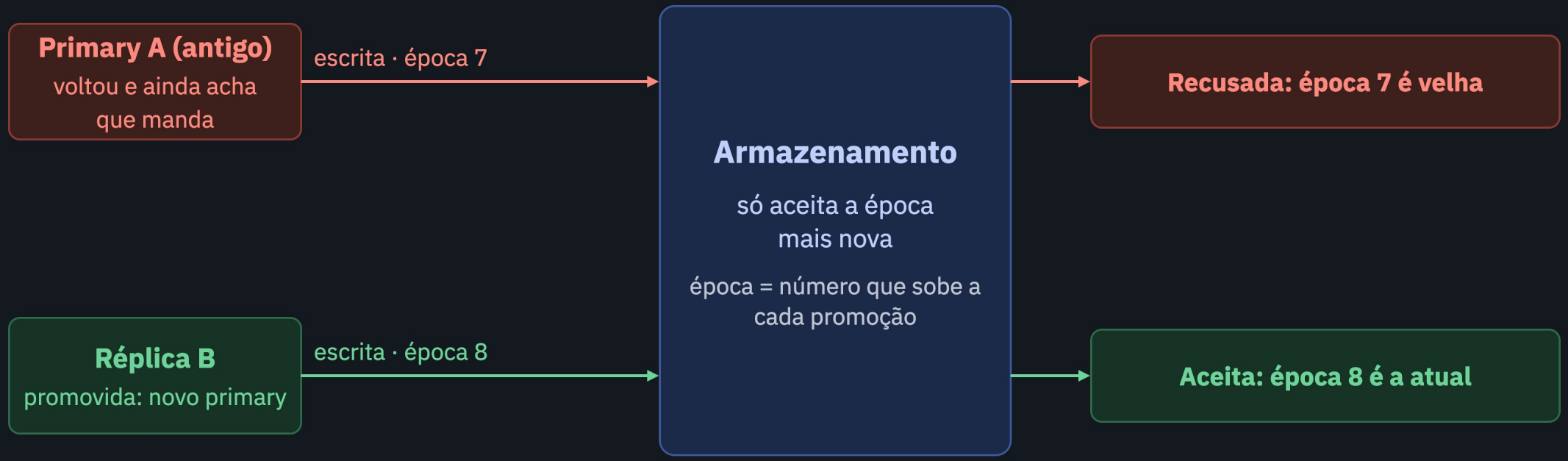
# Fencing: barrar o antigo primary

O QUE É

Tirar a autoridade de escrita do antigo primary. O banco passa a recusar qualquer escrita dele, mesmo que ele tente.

ANALOGIA

É como cancelar o cartão de acesso de quem saiu da empresa: a pessoa até tenta passar na catraca, mas a porta não abre.




**Formas de fazer fencing: isolar ou desligar · revogar credenciais · token de época**

# O antigo primary voltou. Quem pode escrever?




**Split-brain = dois escritores → estados divergentes → efeitos financeiros conflitantes.**

## Fencing: tirar a autoridade de escrita de A

- Isolar ou desligar o antigo primary
- Revogar as credenciais de escrita dele
- Token de época que o armazenamento rejeita se estiver velho
- Confirmar o fencing **antes** de liberar tráfego

## Trocar o routing não basta

Conexões abertas, jobs, consumers e configurações antigas ainda chegam ao antigo primary.

# Perder uma zona não é perder uma região

| COMPONENTE | MÚLTIPLAS ZONAS · MESMA REGIÃO | SEGUNDA REGIÃO · O QUE EXIGE |
|---|---|---|
| **Aplicação** | Pods distribuídos entre zonas | Cluster pronto com imagens e configuração equivalentes |
| **Banco** | Réplica em outra zona, latência baixa | Replicação a distância, geralmente assíncrona: exposição à perda |
| **Kafka** | Réplicas de partição entre zonas | Replicação entre clusters; offsets e ordem precisam de tratamento |
| **Identity** | Instâncias em várias zonas | Emitir e validar tokens sem a região principal |
| **Secrets** | Cofre regional resiliente a zonas | Cópia acessível e rotacionada fora da região principal |
| **Observabilidade** | Coleta em todas as zonas | Enxergar a região perdida de fora dela |

**Kubernetes sozinho não é DR.** Ele reagenda pods; não replica dados, não sincroniza secrets e não decide failover.

**Dependências compartilhadas** — DNS, Identity central, conta, pipeline, cofre de secrets — podem derrubar os dois ambientes.

# Quatro estratégias, quatro apostas

COMPARAÇÃO QUALITATIVA

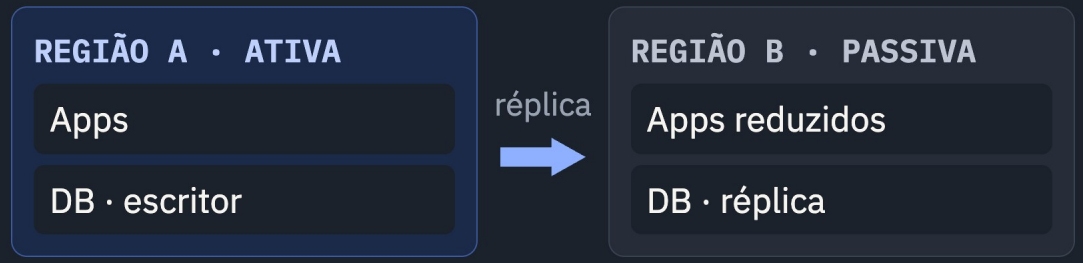
| | Backup / restore | Pilot light | Warm standby | Ativo-ativo |
|---|---|---|---|---|
| PREPARAÇÃO DO AMBIENTE | Só cópias; tudo é recriado | Dados replicados e núcleo mínimo; aplicações desligadas | Ambiente reduzido rodando, escala sob demanda | Duas regiões servindo tráfego |
| RECUPERAÇÃO ESPERADA | Mais lenta; depende do restore | Subir e escalar aplicações | Escalar e redirecionar | Redistribuir carga e ownership |
| CUSTO | ●○○○ | ●●○○ | ●●●○ | ●●●● |
| COMPLEXIDADE | ●○○○ | ●●○○ | ●●●○ | ●●●● |

**Nenhuma estratégia tem RTO/RPO fixo: o resultado depende da implementação e é comprovado em teste.**

# Duas regiões ativas ≠ dois escritores

## Ativo-passivo

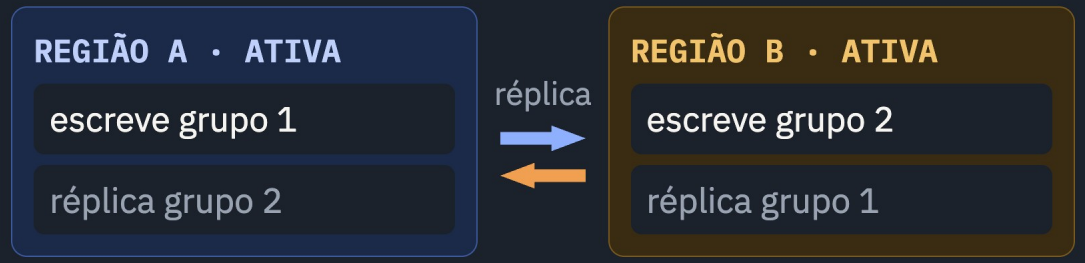
`routing → 100% na região A`




B aguenta a carga toda? Está aquecida? Quanto leva o routing?

## Ativo-ativo com ownership

`clientes do grupo 1 → A` `clientes do grupo 2 → B`

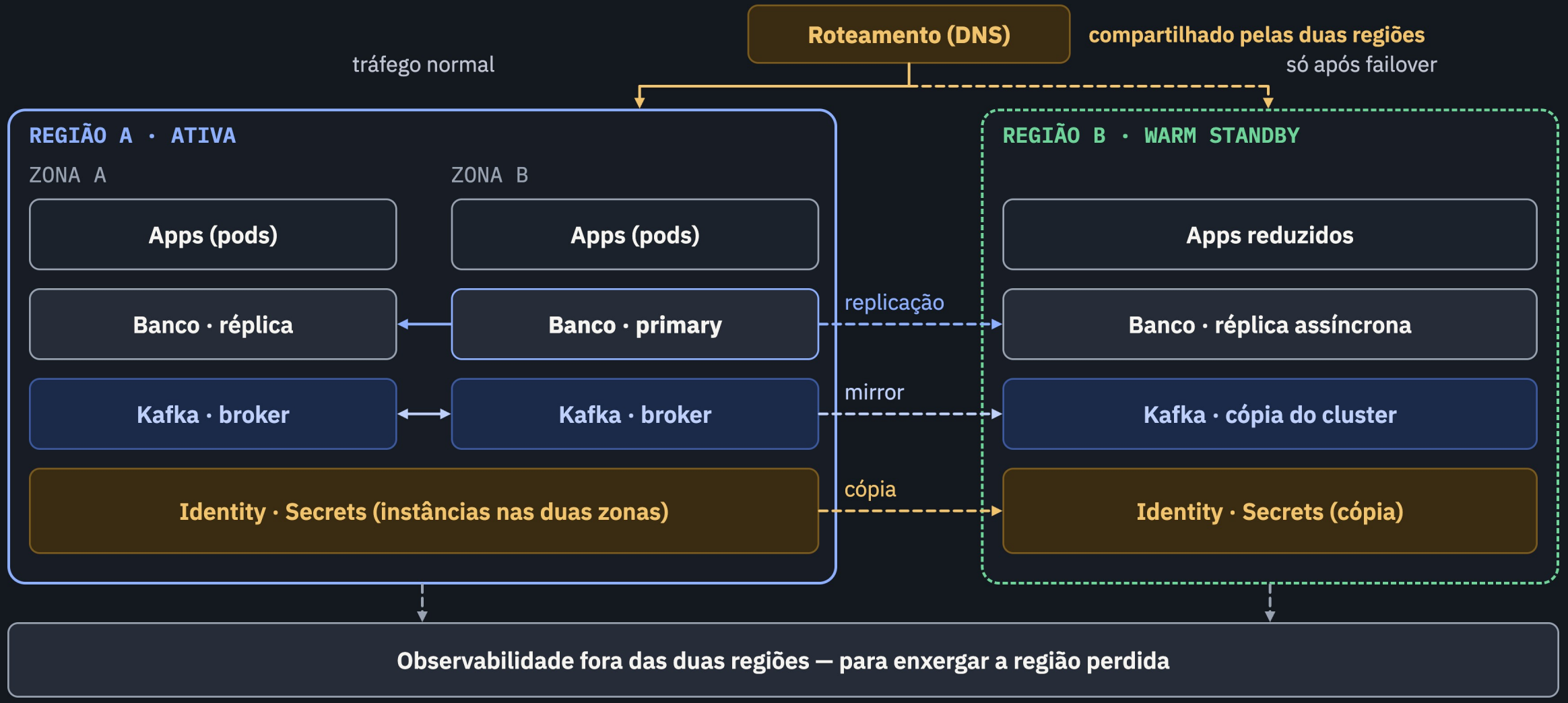



Cada dado tem um **único dono**. Trocar o dono exige fencing.

Replicação · Capacidade de assumir carga · Aquecimento · Routing · Ownership · Consistência · Coordenação das escritas

**Nenhum dos dois modelos entrega RTO ou RPO zero automaticamente.**

# Topologia alvo: multi-zona + warm standby



# Quanto custa essa disponibilidade?

CUSTO TOTAL = MUITO ALÉM DA INFRA

Infraestrutura redundante · Capacidade reservada · Armazenamento · Backups · Replicação · Transferência de dados · Observabilidade · Licenças, se aplicáveis · **Engenharia** · **Plantão** · **Testes e Game Days** · **Manutenção contínua**

Em azul forte: custos recorrentes de pessoas e processo.

| PROPOSTA | CUSTO | COMPLEXIDADE | RECUPERAÇÃO |
|---|---|---|---|
| **A · Menor custo**<br>Backup/restore em outra região | ●○○ | ●○○ | mais demorada |
| **B · Secundário preparado**<br>Pilot light ou warm standby | ●●○ | ●●○ | intermediária |
| **C · Múltiplas regiões**<br>Ativo-ativo com ownership | ●●● | ●●● | potencialmente mais rápida |

Compare o investimento com o impacto evitado e com as metas por capacidade.

**Sem valores de propósito: o preço depende de provedor, região, volume, arquitetura e equipe.**

# Três fontes, três versões da verdade

EXEMPLO DIDÁTICO

| OPERAÇÃO | TECH PIX · ESTADO RECUPERADO | PROVIDER | LEDGER | REENVIAR? |
|---|---|---|---|---|
| P-101 | COMPLETED | Liquidado | Lançado | ? |
| P-102 | PROCESSING | Liquidado | Sem lançamento | ? |
| P-103 | UNKNOWN | Rejeitado | Sem lançamento | ? |
| P-104 | AUSENTE perdida no RPO | Liquidado | Sem lançamento | ? |
| P-105 | PROCESSING | Sem registro | Sem lançamento | ? |

**Ausência de confirmação local não comprova falha do pagamento.**

**Quais podem ser reenviadas com segurança?**

# Reabrir só depois de provar a integridade




INVARIANTES A VERIFICAR

- ✓ Cada pagamento tem **no máximo um** efeito financeiro
- ✓ Lançamentos do Ledger consistentes: débitos = créditos
- ✓ Toda operação está conhecida ou em uma lista explícita de reconciliáveis

DIVERGÊNCIA: INVESTIGAR, NÃO REEXECUTAR

1. Consultar o provider pelo ID externo ou chave de idempotência
2. Pôr operações ambíguas em quarentena
3. Corrigir estado e Ledger com decisão registrada

# Brokers de volta ≠ efeitos recuperados




**Mensagem perdida**
Outbox pendente precisa ser republicado

**Offset adiante do estado**
Evento consumido, efeito perdido no restore

**Replay duplica**
Consumidor idempotente torna seguro

DEGRADAÇÃO CONTROLADA

- manter: pagamento
- manter: Ledger
- manter: consulta de status
- adiar: Notification
- adiar: Analytics
- adiar: relatórios

RISK INDISPONÍVEL: POLÍTICA EXPLÍCITA

Definida antes do incidente. Sem persistência e controle seguros, **novos pagamentos podem precisar ser interrompidos**.

# Runbook: quem faz o quê, e quando parar

| # | PASSO | RESPONSÁVEL |
|---|---|---|
| 01 | **Declarar o incidente** | Comandante do incidente |
| 02 | **Conter: parar retries e jobs** | Operação |
| 03 | **Autorizar o failover** | Pessoa com autoridade definida |
| 04 | **Fencing e routing** | Plataforma |
| 05 | **Recuperar** | Dados e plataforma |
| 06 | **Validar** | Times dos serviços |
| 07 | **Reconciliar** | Pagamentos e financeiro |
| 08 | **Comunicar (contínuo)** | Comunicação |
| 09 | **Reabrir progressivamente** | Comandante do incidente |

INTERROMPER O PROCEDIMENTO SE…

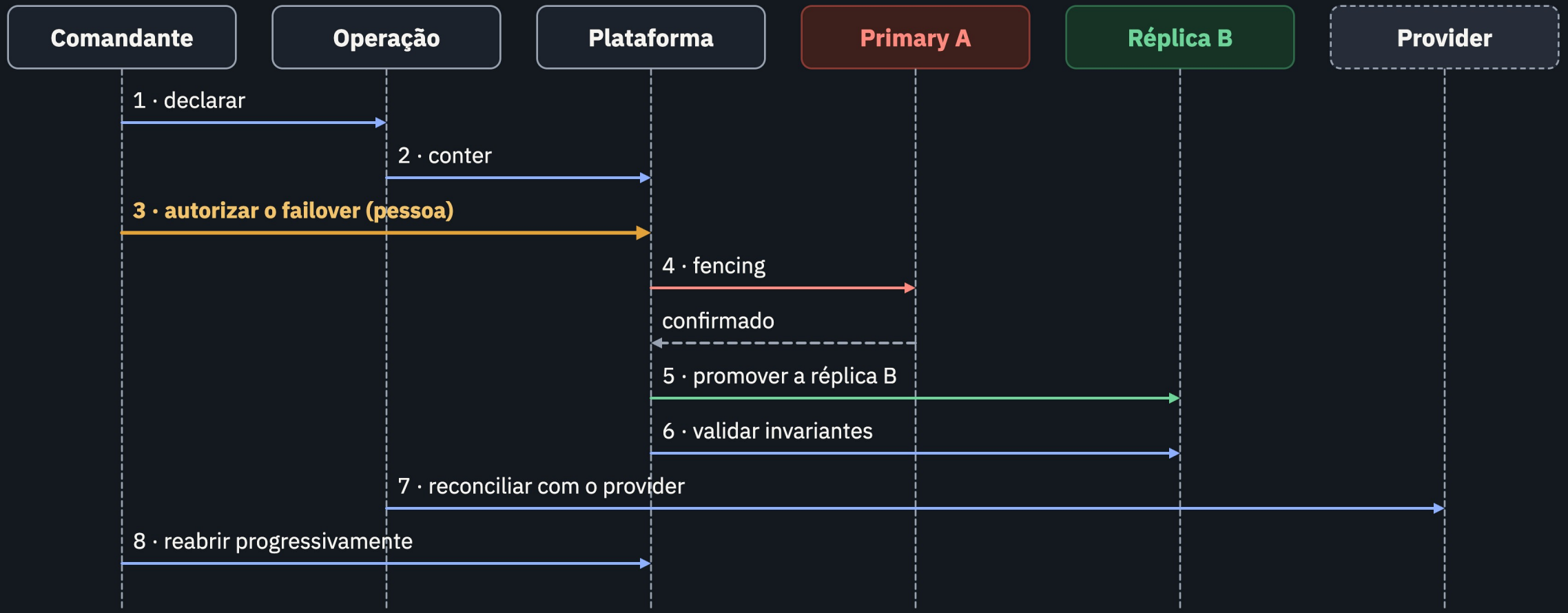
- fencing não confirmado
- atraso da réplica acima do RPO aceito
- duplicação detectada
- divergência sem explicação

RETORNO À REGIÃO ORIGINAL

Outra operação planejada: ressincronizar, nova janela, mesmo rigor de validação.

**IA recomenda. Pessoas autorizam.**

# Failover passo a passo: quem fala com quem




**INTERROMPER SE: fencing não confirmado · atraso da réplica acima do RPO · duplicação detectada · divergência sem explicação**

# Game Day: provar com evidências

20 min

CENÁRIO

- Primary indisponível
- Réplica atrasada
- Provider ainda processando

MISSÃO

1. Decidir
2. Recuperar
3. Reconciliar
4. Validar

Ambiente de laboratório preparado e ensaiado antes da aula.

| RELATÓRIO DA EQUIPE | SÓ MEDIÇÕES |
|---|---|
| Detecção | __h__ |
| Recuperação validada | __h__ |
| Perda observada | intervalo · qtd. |
| Transações afetadas | lista |
| Duplicações | qtd. · quais |
| Divergências | resolvidas · controladas |
| Invariantes | ok ou falha, cada um |

# Arquitetura final da Tech Pix, camada por camada

**1 · FLUXO FINANCEIRO**

Gateway → Payment → Fraud/Risk → Provider externo → Ledger

**2 · DADOS E EVENTOS**

Payment DB · Ledger DB · Risk DB · Outbox → Kafka → Notification · Analytics

**3 · SEGURANÇA**

Identity · mTLS entre serviços · Secrets replicados

**4 · OBSERVAR E RECONCILIAR**

Traces · métricas · logs fora da região · Reconciliação com provider e Ledger

**5 · RECUPERAÇÃO**

Multi-zona · Warm standby na 2ª região · Backups testados · Fencing · Runbook + Game Days

## CADA ELEMENTO PRECISA TER

- **Problema** que o motivou
- **Decisão** tomada
- **Custo** assumido
- **Validação** que o comprova

## HIPÓTESES E LIMITAÇÕES

- Provider aceita consulta por chave de idempotência
- RTO de 5 min ainda não medido
- Provider externo segue como dependência única
- Kafka entre regiões ainda não testado em Game Day

# Nossa proposta para os investidores

METAS · AINDA NÃO COMPROVADAS

### CAPACIDADES PROTEGIDAS

- Iniciar pagamento
- Consultar status
- Registrar no Ledger

### SLO

Pagamentos concluídos corretamente ÷ válidos, em 30 dias

**Alvo: [definir com a turma]**

### RTO / RPO PRETENDIDOS

- Payment, Ledger: 5 min · RPO [a definir]
- Notification: 2 h
- Analytics: 24 h

### ESTRATÉGIA

Multi-zona + warm standby em 2ª região para o fluxo financeiro. Backup/restore para Analytics.

### CUSTOS ENVOLVIDOS

2ª região reduzida, replicação e transferência, observabilidade externa, plantão, Game Days

### RISCOS RESIDUAIS

Perda dentro do RPO · provider fora do ar · dependência compartilhada · erro humano

### EVIDÊNCIAS NECESSÁRIAS

Restore medido · Game Day dentro da meta · zero duplicação · reconciliação completa

**Defendam — ou troquem — esta escolha.**

**Não prometemos “nunca offline”. Prometemos metas claras, riscos conhecidos e evidências medidas.**

# Arquitetura é uma decisão contínua

| DECISÃO | O QUE GANHAMOS | O QUE PAGAMOS |
|---|---|---|
| **Começamos modulares** | + simplicidade | – deploy acoplado |
| **Crescemos** | + clientes | – mais responsabilidade |
| **Distribuímos** | + escala independente | – rede e consistência |
| **Protegemos** | + confiança entre serviços | – identidade e secrets |
| **Instrumentamos** | + visibilidade | – volume de dados |
| **Tratamos falhas** | + contenção | – configuração delicada |
| **Planejamos continuidade** | + recuperação ensaiada | – custo recorrente |

> “Arquitetura é tomar decisões sob restrições, medir suas consequências e evoluir quando a realidade muda.”

A seguir: 20 minutos de Game Day